

List

- List is an ordered sequence of elements, where order of elements matter.
- Elements of a list may be constant, variable or list themselves.
- List can contain an unlimited number of elements.
- All the objects in a list must^{be} of the same type, but may be very complex.
- In LISP, the only data structure available is List apart from constants.
- The list can be bifurcated as Head and Tail.

The notation is [Head | Tail].

Head — Contains the first element of a list

Tail — It's a list which contains all the elements except the Head.

Example

List

Head

Tail

[a, b, c]

a

[b, c]

[the, [cat, sat]]

the

[[cat, sat]]

[]

None

None

Matching of Lists

List 1

List 2

Instantiation

[x, y, z]

[john, likes, fish]

X = john

Y = likes

Z = fish

[cat]

[x | y]

X = cat

Y = []

[x, y | z]

[marry, likes, wine]

X = marry

Y = likes

Z = [wine]

[[the, y] | z]

[[~~x~~, here], [is, here]]

X = the

Y = here

Z = [[is, here]]

[vale, horse]

[horse, x]

Don't match

Declaring list in PROLOG

domains

list = symbol *

predicates

member (symbol, list)

clauses

member (x, [x|_]).

member (x, [_|T]) :- member(x, T).

goal: member (susan, [tom, bill, susan])

Call: member (susan, [tom, bill, susan])

Fail: member (susan, [tom, bill, susan])

Redo: member (susan, [tom, bill, susan])

Call: member (susan, [bill, susan])

Fail: member (susan, [bill, susan])

Redo: member (susan, [bill, susan])

Call: member (susan, [susan])

Ret: member (susan, [susan])

Ret: member (susan, [bill, susan])

Ret: member (susan, [tom, bill, susan])

True

Goal: writelist([a, b, c]).

a

b

c

Yes

domains

list = symbol *

predicates

writelist(list)

clauses

writelist([Head | Tail]) :- write(Head), nl,
writelist(Tail).

Output:

a

b

c

No

How to eliminate "No" from the answer?

Soln: By writing writelist[] as last clause.

- Even, if we write writelist([]) over the given clause, still answer doesn't change.

In Recursive Definitions, we must look for:

(a) Left Recursion

This arises when a rule causes the invocation of a goal that is essentially equivalent to the original goal that caused the rule to be used.

Thus, if we defined:

mother (lilly, adam).

person (X) :- person (Y), mother (X, Y).

person (adam).

Goal: person (Z).

Problem ?

Backtracking is not possible, because in

order to backtrack a clause must fail.

Solⁿ : If we put person (adam) over a rule, then PROLOG has chance to look for finding solutions.

Heuristics: Put facts over rules in DB.

(b) Circular Definitions

$$\text{parent}(x, y) \text{ :- child}(y, x).$$

$$\text{child}(x, y) \text{ :- parent}(y, x).$$

Example 2

domains

$$\text{list} = \text{integer}^*$$

predicates

$$\text{testlist}(\text{list})$$

clauses

$$/* c1 */ \quad \text{testlist}([_ | \text{Tail}]) \text{ :- testlist}(\text{Tail}).$$

$$/* c2 */ \quad \text{testlist}([]).$$

$$/* \text{goal: testlist}([_ | \text{Tail}]) \text{ :- testlist}(\text{Tail}). */$$

Yes

$$\text{goal: testlist}([1, 2]).$$

Yes

$$\text{goal: testlist}(x).$$

Stack overflow

If we interchange $c1$ and $c2$, then O/P will be

$$X = []$$

Appending a list

append([], L, L).

append([X | L1], L2, [X | L3]) :-

append(L1, L2, L3).

Goal: append([a, b, c], [d, e, f], X)

X = [a, b, c, d, e, f]

1 solution.

Goal: append(X, [a, b, c], [d, e, a, b, c])

X = [d, e]

1 solution.

Execution Trace:

Call Goal: append(["a", "b"], ["c", "d"], ~~X~~)

Redo : append(["a", "b"], ["c", "d"], -)

Call: append(["b"], ["c", "d"], -)

Redo: append(["b"], ["c", "d"], -)

Call: append([], ["c", "d"], -)

Ret: append([], ["c", "d"], ["c", "d"])

Ret: append(["b"], ["c", "d"], ["b", "c", "d"])

Ret : append(["a", "b"], ["c", "d"], ["a", "b", "c", "d"])

["a", "b", "c", "d"]

Reversing a List

(6) (41)

domains

list = integer *

predicates

reverse (list, list, list)

clauses

newreverse ([], Inputlist, Inputlist).

newreverse ([Head | Tail], List1, List2) :-

newreverse (Tail, [Head | List1], List2).

goal: reverse ([1,2], [], Z)

Z = [2,1]

| Solution

or if the goal is
reverse (List1, Z)
?- newreverse (List1,
[], Z).

{ goal: reverse (Z, [], [1,2]) /* Error */ }

Call: reverse ([1,2], [], z)

Redo: reverse ([1,2], [], z)

Call: reverse ([2], [1], —)
List2

Redo: reverse ([2], [1], —)

Call: reverse ([], [2,1], —)

Ret: reverse ([], [2,1], [2,1])

Ret: reverse ([2], [1], [2,1])

Ret: reverse ([1,2], [], [2,1])

Rotati ~~Right~~ Left

reverse ([H | T], X) :- append (T, [H], X)

domains

list = integer *

predicates

last-element (list, integer)

clauses

last-element ([z], x) :- z = x.

This can be written as

last-element ([x], x).

last-element ([_ | Tail], x) :-

last-element (Tail, x).

Goal: last-element ([1,2,3], x)

x = 3

1 solution

Goal: last-element ([1,2,3], 3)

Yes

Goal: last-element ([], x)

No solution

What if we change 1st predicate: last-element (z, z).

With this change, if the goal is last-element ([1,2,3], x)

x = [1,2,3]

x = [2,3]

x = [3]

x = []

4 sol^{ns}

[Here, the declaration of the list should be
last-element (list, list)]

Execution Trace of the last element with above change:

Call: 1-element([1,2,3], -)

Ret: 1-element([1,2,3], [1,2,3])

↑ First Solⁿ ← First Clause

Redo: 1-element([1,2,3], -)

← Second Clause Call

Call: 1-element([2,3], -)

Ret: 1-element([2,3], [2,3])

← First Clause success

Ret: 1-element([1,2,3], [2,3])

↑ 2nd Solⁿ

← Second Clause success

Redo: 1-element([2,3], -)

← Second clause

Call: 1-element([3], -)

Ret: 1-element([3], [3])

← First Clause

Ret: 1-element([2,3], [3])

Ret: 1-element([1,2,3], [3])

↑ 3rd Solution

Redo: 1-element([3], -)

Call: 1-element([], -)

Ret: 1-element([], [])

← First Clause

Ret: 1-element([3], [])

Ret: 1-element([2,3], [])

Another Version of L-E

Last element using append

last-element(L, R):-
append(-, [R], L).

Ret: 1-element([1,2,3], [])

↑ 4th Solⁿ

Redo: 1-element([], -) ← second clause

fail: 1-element([], -)

4 Solutions

Finding n th element of a list

domains

list = integer *

predicates

nthelement (list, integer)

clauses

nthelement ([Head | _], 1) :-

write (Head), nl.

nthelement ([_ | Tail], N) :-

NN = N - 1,

nthelement (Tail, NN).

goal : nthelement ([1, 2, 3], 3)

3

Yes

goal : nthelement ([1, 2, 3], 4)

No

Note: This works for N to be > 1 .

A slight modification can be made so that it works for any integer N (Putting $N > 0$ as a condition)

Finding Maximum of a list

Goal: maximum ([5, 2, 7, 3], x)

 $x = 7$ 1 solnV1

/*c3*/ max ([], Max, Max).

/*c1*/ max ([Head | Tail], Max, R) :-

Head > Max, max (Tail, Head, R).

/*c2*/ max ([Head | Tail], Max, R) :-

Head <= Max, max (Tail, Max, R).

If we put 'cut' in c1, Then Head <= Max is not required. 'cut' may be put as the last element in c1.

V2

max ([x], x).

max ([Head | Tail], Head) :-

max (Tail, x),

Head > x.

max ([Head | Tail], x) :-

max (Tail, x),

Finding length of a list

goal: length ([2, 1, 6], x)

$x = 3$ is solⁿ

length ([], 0).

length ([_ | T], N) :- length (T, NN),

$N = NN + 1.$

$NN = N - 1$ won't work

("Free Variable in Expression" error)

$N = NN + 1$, length (T, NN)

won't work and give same error as above.

Finding Sum of first N Natural Numbers

sum (0, 0).

sum (N, Sum) :- $N > 0$, $N1 = N - 1$,

sum (N1, R1), $Sum = R1 + N.$

OR

Without Recursion

sum (N, Sum) :- $Sum = N * (N + 1) / 2.$

Finding odd & even elements separately and framing lists, i.e. Split the input lists into two lists, 1 containing odd elements and the other containing even elements.

goal: $oe([5, 3, 2, 1], X, Y).$

$X = [5, 3, 1]$

$Y = [2]$

1 solution.

$oe([], [], []).$

$oe([H|T], [H|T1], T2)$

$:- H \bmod 2 \neq 0,$

$oe(T, T1, T2).$

$oe([H|T], T1, [H|T2])$

$:- H \bmod 2 = 0,$

$oe(T, T1, T2).$